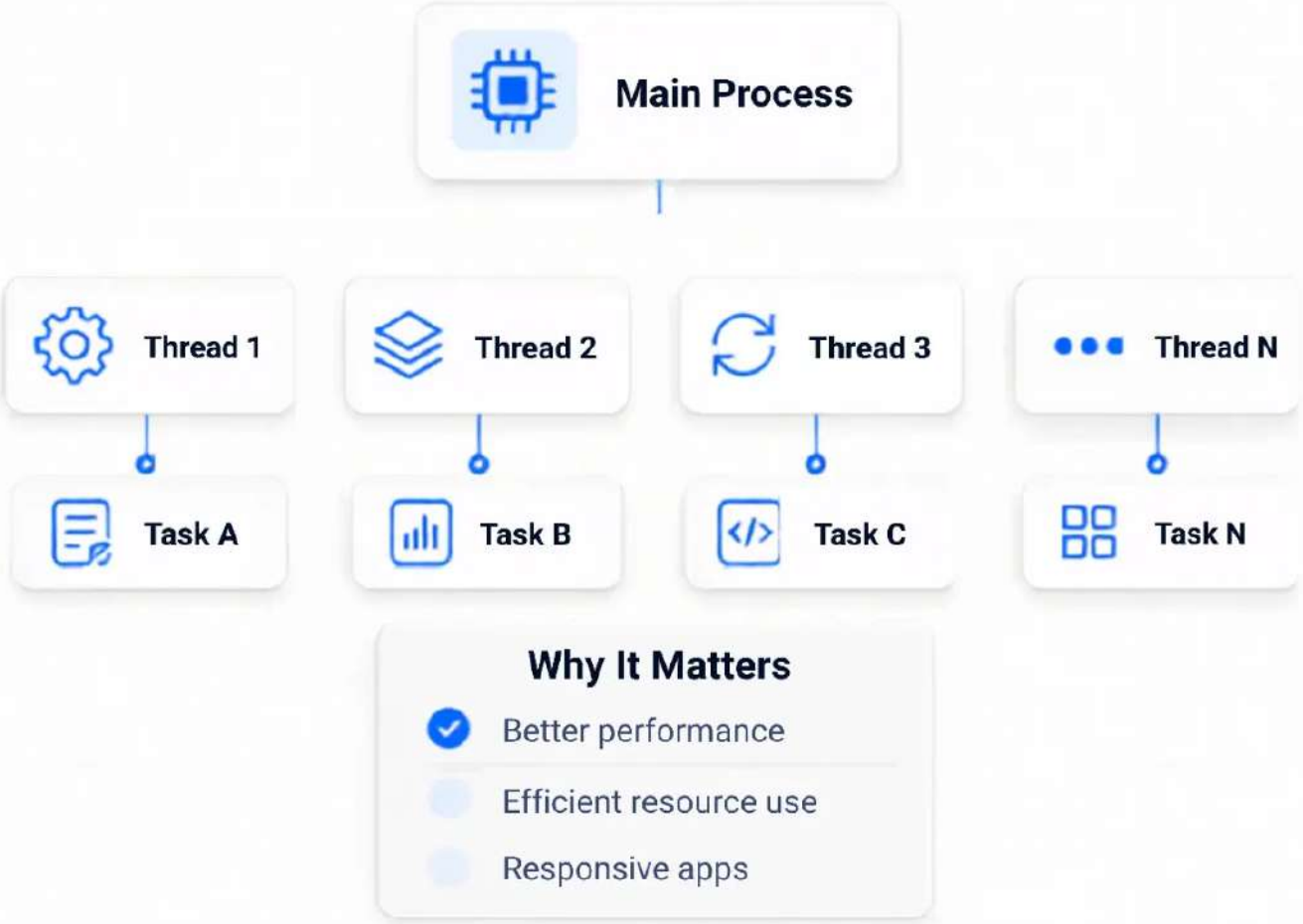
Full Stack Web Dev
@web_code_wizard

Multithreading In Java Explained

How Java runs multiple tasks at the same time.



One process can run multiple threads.

Full Stack Web Dev
@web_code_wizard

What Is A Thread?

A thread is the **smallest unit of execution** inside a process.

Easy Analogy

One kitchen can prepare multiple dishes at the same time — one process can run multiple threads.

Independent
Each thread runs its own task.

Concurrent
Multiple threads work together.

Efficient
Better CPU usage and performance.

One process can have multiple threads working together.

Java Multithreading

Process vs Thread

Understand the real difference in Java.

Process

An independent program in execution.

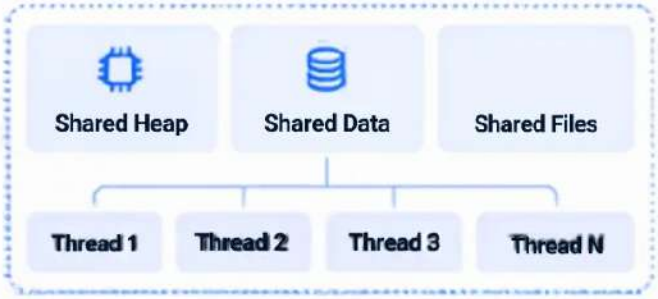
- Has its own memory space
- Heavyweight compared to threads
- Communication is slower
- Runs independently

Examples: Browser, VS Code, MySQL

VS

Thread

A lightweight unit of execution within a process.



- Shares memory with other threads
- Lightweight
- Faster communication
- Best for parallel tasks inside one process

Example: Multiple threads inside one Java app

Key takeaway

Multiple threads inside one process can improve speed, responsiveness, and CPU utilization.

Faster execution | **Better performance** | **More responsive apps**

Why Multithreading?

Java runs **multiple tasks** at the same time.

01

Faster Execution

Multiple tasks can run together, reducing total execution time.

02

Better CPU Use

Keeps the CPU busy while other threads are waiting.

03

Responsive Apps

Apps stay responsive while background tasks keep running.

04

Efficient Resources

Threads share memory inside one process, making execution lighter.

Real-World Uses

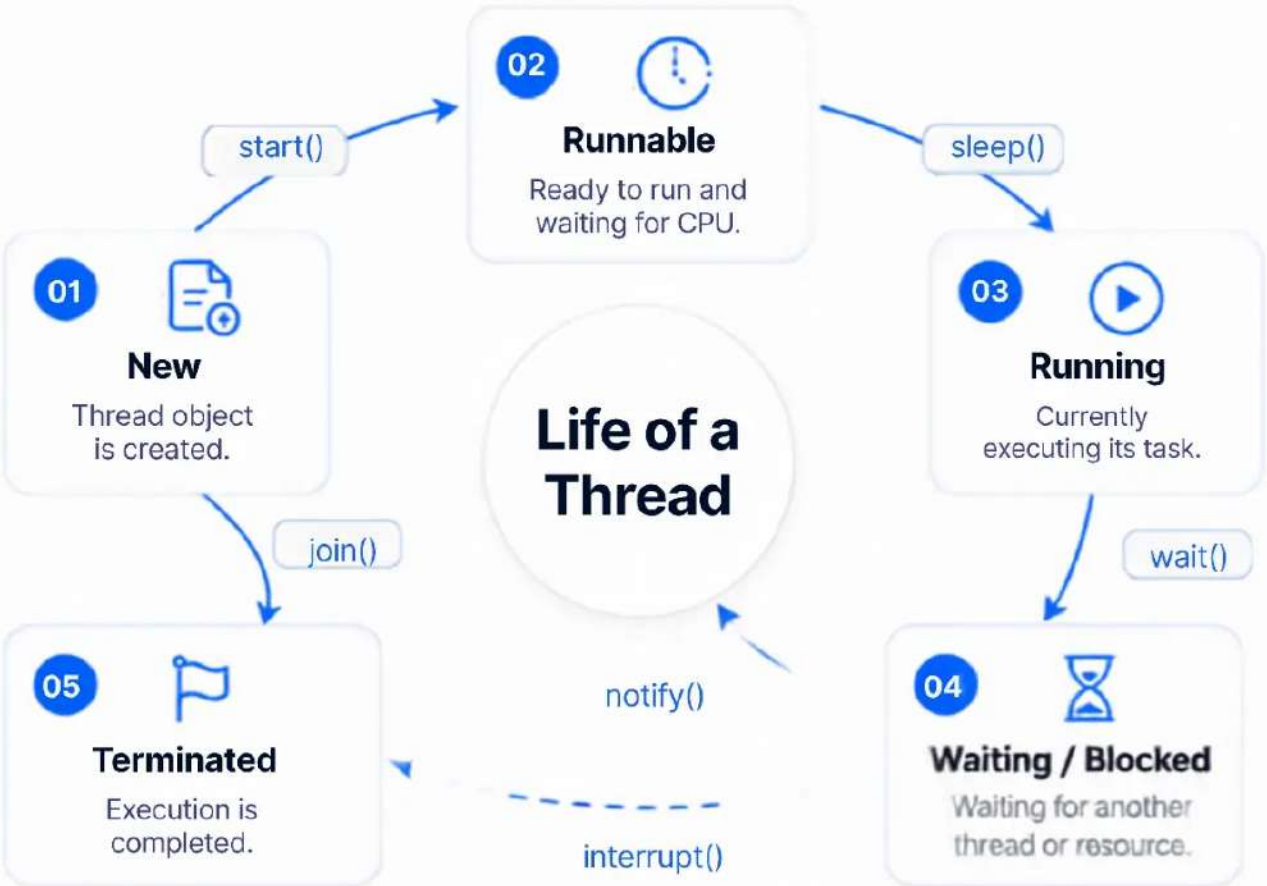
- Web Servers
- File Downloads
- Chat Apps
- Games
- Media Players

Multithreading = speed + efficiency + better user experience.

Full Stack Web Dev
@web_code_wizard

Thread Lifecycle

A thread goes through **different states** during its lifetime.



Common Methods

start() sleep() join() notify() interrupt()

Why It Matters

Understanding lifecycle helps you write better and more reliable multithreaded code.

Better lifecycle understanding = more reliable Java code.

Full Stack Web Dev
@web_code_wizard

Race Condition

When multiple threads access the **same data** at the same time.

Thread 1
Withdraw ₹700

1. Read balance: ₹1000
2. Calculate: 1000 - 700
3. Write balance: ₹300

Shared Bank Account

Balance:
₹1000

Thread 2
Withdraw ₹500

1. Read balance: ₹1000
2. Calculate: 1000 - 500

Write balance: ₹500

Incorrect Result
Expected balance: ₹-200
Actual balance: ₹300 or ₹500

Why It Happens

Threads can interleave their execution, so the final result depends on timing.

Solution

Use **synchronization** or **locks** so only one thread accessing shared data at a time.

Shared data without coordination can create unpredictable results.

Synchronization In Java

Only one thread can access shared data at a time.

The Problem

Without synchronization, multiple threads can update shared data at the same time.

Result can become inconsistent.

The Solution

Synchronization locks the critical section so only one thread executes it at a time.

Critical Section

Code that accesses shared data.

Protected by synchronized

```
class BankAccount {
   private int balance = 1000;

   public synchronized void withdraw(int amount) {
      balance -= amount;
   }
}
```

How It Works

synchronized protects the method

Only one thread enters at a time

Other threads wait for the lock

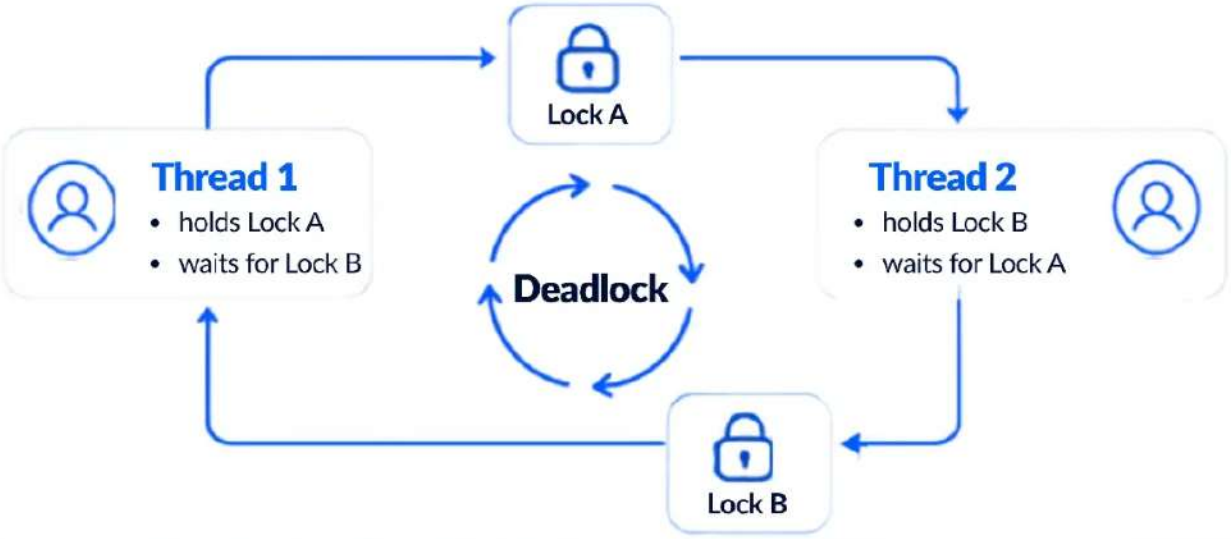
Shared data stays safe

Synchronization = *consistency + safety + reliable results.*

Full Stack Web Dev
@web_code_wizard

Deadlock In Java

A deadlock happens when threads **wait forever** for each other's resources.



What is Deadlock?

A deadlock occurs when two or more threads are **blocked forever** because each thread is **waiting** for a **resource** held by another thread.

Real-Life Example

Two cars block each other on a narrow road.
Both wait for the other to move first.

4 Conditions for Deadlock

01 **Mutual Exclusion** — Resources cannot be shared.

02 **Hold and Wait** — A thread holds one resource and waits for another.

03 **No Preemption** — Resources cannot be taken forcibly.

04 **Circular Wait** — Threads wait in a circular chain.

How to Prevent It

Lock resources in a fixed order

Avoid nested locks when possible

Use tryLock() with timeout

Keep critical sections small

Deadlock = threads waiting forever. Avoid circular waits.

Full Stack Web Dev
@web_code_wizard

Thread Safety In Java

Thread-safe code behaves correctly when **multiple threads** access it.

What is Thread Safe?

A class is **thread-safe** when it works correctly even when multiple threads access **shared data** at the same time.

Not Thread-Safe

```
class Counter
{ int count 0;

   void increment ( count ++
   // not thread safe
 }
```

If two threads call increment() at the same time, one update may be lost.

Thread-Safe

```
class counter {
private int count 0;

   public synchronized void
   increment { count+ ]
   }
```

synchronized allows only one thread to update at a time.

Why It Matters

- Prevents data corruption
- Keeps program behavior predictable
- Makes code reliable in concurrent systems

Best Practices

- Use synchronization wisely
- Keep critical sections small
- Prefer thread-safe classes when possible
- Test with multiple threads

Thread safety = correct results when many threads work together.

Full Stack Web Dev
@web_code_wizard

Best Practices For Java Multithreading

Write efficient, **safe**, and **maintainable** multithreaded code.

01

Keep Tasks Small

Make each thread handle one focused job.

02

Avoid Shared State

Reduce shared mutable data when possible.

03

Use Synchronization

Protect shared resources with locks or synchronized blocks.

04

Use High-Level APIs

Prefer ExecutorService, Semaphore, and concurrent utilities.

05

Avoid Deadlocks

Lock in a fixed order and avoid nested locks.

06

Thread-Safe Collections

Use ConcurrentHashMap, CopyOnWriteArrayList, or BlockingQueue.

07

Handle Interrupts

Respect interruptions and restore interrupt status.

08

Don't Swallow Errors

Log exceptions and handle them properly.

09

Test Under Load

Check real concurrency scenarios before production.

10

Monitor & Tune

Track thread pools, CPU, memory, and performance.

Remember

Good multithreading balances performance, correctness, and maintainability.

Write better threads. Build better applications.